
TALLER DE FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL

Proyecto 2

Autores:

Maximiliano Gallardo Calvo

Brandon Recabarren

Pablo Contreras

Fecha Entrega:

Domingo 4 de octubre de 2026

Índice

Contenido

Dominio y justificación	2
Implementación del Chatbot	2
Set de consultas	3
Análisis de desempeño	5
Conclusiones y mejoras	6
Bibliografía	6

Dominio y justificación

Para el dominio de este proyecto, se utilizó el videojuego *Pokémon* como base. Dado que Pokémon se basa en un sistema de múltiples reglas y relaciones complejas pero claras, es un excelente dominio para ser representado en LPO [1]. Para su representación en LPO, se formula como:

- **Dominio:** *Pokémones, entrenadores y tipos.*
- **Modelamiento del conocimiento:** *pokemon/1, tipo/2, entrenador/1, tiene/2, fuerte_contra/2 (predicado/argumentos o aridad)*
 - **Reglas:** *debil_contra, doblemente_debil, entrenador_vulnerable*

Antes de implementar estas reglas en Prolog, se formalizan en lógica de primer orden. Por ejemplo, la regla de debilidad se expresa como:

$$\forall x \forall t (\text{DebilContra}(x,t) \leftrightarrow \exists td (\text{Tipo}(x,td) \wedge \text{FuerteContra}(t,td)))$$

Es decir: para todo pokémon x y todo tipo de ataque t, x es débil contra t si y solo si existe un tipo td tal que x es de tipo td y t es fuerte contra td. De manera análoga, la regla de vulnerabilidad de un entrenador se formaliza como:

$$\forall e \forall t (\text{EntrenadorVulnerable}(e,t) \leftrightarrow \exists x (\text{Tiene}(e,x) \wedge \text{DebilContra}(x,t)))$$

Para mejorar la visualización del modelo, se visualiza en la *Tabla 1* la representación mencionada:

<i>Predicado</i>	<i>Aridad</i>	<i>Representa</i>
<i>pokemon</i>	1	<i>Pokémon único</i>
<i>tipo</i>	2	<i>Tipo asociado a un Pokémon</i>
<i>entrenador</i>	1	<i>Entrenador Pokémon</i>
<i>tiene</i>	2	<i>Indica que Pokémon está asociado a cada entrenador</i>
<i>fuerte_contra</i>	2	<i>Indica que tipo es fuerte contra otro</i>

Implementación del Chatbot

Como objetivo, se buscó lograr una interacción directa con el usuario y una capacidad de modelamiento más avanzada utilizando la base de conocimiento disponible por medio de la

creación de un chatbot en Prolog. El chatbot propuesto cuenta con las siguientes características:

- Bucle e inicio de conversaciones por lecturas de la entrada del usuario
- Uso de tokens por medio de separación de Strings y conversión a átomos
- Conversión de la pregunta a una lista de palabras
- Uso de coincidencia de patrones con *member* y *append* para reconocimiento de palabras claves
- Múltiples interpretaciones según el átomo identificado introducido por el usuario
- Respuesta adaptada a la interpretación respectiva

Set de consultas

Posterior a la representación del dominio, se realiza un conjunto de 20 consultas con sus respuestas respectivas dadas por el chatbot dentro de SWI- *Prolog*:

Nº	Pregunta	Consulta	Resultado
1	¿Es pikachu un pokemon?	pokemon(pikachu).	Si, pikachu es un Pokemon
2	¿Es pikachu de tipo eléctrico?	tipo(pikachu, electrico).	El pokemon pikachu es de tipo: [electrico]
3	¿Qué tipo(s) es charizard?	tipo(charizard, Tipo).	El pokemon charizard es de tipo: [fuego,volador]
4	¿Qué pokemon son de tipo fuego?	tipo(P, fuego).	Los pokemon de tipo fuego son: [charmander,charmeleon,charizard,ho_o h,flareon]
5	¿Qué pares pokemon-tipo existen registrados?	tipo(P, T).	Los pares pokemon-tipo registrados son: [pikachu-electrico,raichu-electrico, ...] (52)
6	¿Qué pokemones tiene ash?	tiene(ash, P).	El entrenador ash tiene a los siguientes pokemon: [pikachu,charizard,bulbasaur,squirtle,sno rlx]
7	¿Quién tiene a gyarados?	tiene(E, gyarados).	Los entrenadores que tienen a gyarados son: [red,misty,lance]
8	¿Es eléctrico fuerte contra agua?	fuerte_contra(electrico, agua).	Si, electrico es fuerte contra agua

9	¿Contra qué tipos es fuerte el tipo fuego?	fuerte_contra(fuego, Tipo).	El tipo fuego es fuerte contra: [planta,hielo,bicho,acero]
10	¿Es charizard débil contra agua?	debil_contra(charizard, agua).	Si, charizard es debil contra agua
11	¿Contra qué tipo de ataque es débil gengar?	debil_contra(gengar, TipoAtaque).	gengar es debil contra los tipos: [fantasma,siniestro,tierra,psiquico]
12	¿Qué pokemon son débiles contra tipo hielo?	debil_contra(P, hielo).	Los pokemon debiles contra el tipo hielo son: [charizard,bulbasaur,ivysaur,venusaur,pidgey,pidgeotto,onix,geodude,golem,lugia,ho_oh,dragonite,dragonite,gyarados]
13	¿Es charizard doblemente débil contra roca?	doblemente_debil(charizard, roca).	Si, charizard es doblemente debil contra roca
14	¿Qué pokemon son doblemente débiles y contra qué tipo?	doblemente_debil(P, TipoAtaque).	Pares pokemon-tipo doblemente debiles: [charizard-roca,onix-agua,onix-planta,geodude-agua,geodude-planta,golem-agua,golem-planta,ho_oh-roca,dragonite-hielo,gyarados-electrico]
15	¿Es brock vulnerable a tipo agua?	entrenador_vulnerable(brock, agua).	Si! El entrenador brock es vulnerable a ataques de tipo agua
16	¿A qué tipos de ataque es vulnerable misty?	entrenador_vulnerable(misty, TipoAtaque).	El entrenador misty es vulnerable a los tipos de ataque: [electrico,planta,hielo,roca]
17	¿Qué pokemon de tipo fuego tiene ash?	tipo(P, fuego), tiene(ash, P).	El entrenador ash tiene estos pokemon de tipo fuego: [charizard]
18	¿Hay algún entrenador registrado que tenga a snorlax?	entrenador(E), tiene(E, snorlax).	Los entrenadores que tienen a snorlax son: [ash,red]
19	¿Es verdad que brock no tiene a pikachu?	\+ tiene(brock, pikachu).	Es verdad, brock no tiene a pikachu

20	¿Qué otro pokemon comparte un tipo con charizard?	tipo(charizard, T), tipo(P2, T), P2 \= charizard.	Pokemon que comparten un tipo con charizard: [charmander,charmeleon,ho_oh,flareon,pidgey,pidgeotto,lugia,dragonite,gyarados]
----	---	---	---

Análisis de desempeño

Al realizar un análisis del desempeño dentro del modelo, se observa que el chatbot responde de manera correcta a las 20 consultas del dominio cuando estas se formulan utilizando las palabras clave esperadas por cada regla de interpretación.

Fortalezas del modelo:

- El modelo entrega respuestas deterministas y exactas gracias a que se basa en la lógica de primer orden, evitando inventar datos
- Facilidad de rastreo de la ruta de inferencia de cada respuesta dentro del código mediante el orden de evaluación

En si, sistema logra distinguir correctamente entre preguntas que requieren una respuesta booleana (por ejemplo, “¿es charizard débil contra agua?”) y preguntas abiertas que requieren enumerar todos los elementos que cumplen una condición (por ejemplo, “¿qué pokémon son débiles contra hielo?”), además de preguntas que combinan dos entidades nombradas, como la comparación de ventaja entre dos pokémon o la búsqueda de pokémon que comparten un tipo con otro.

Debilidades del modelo:

- Cuenta con rigidez sintáctica, el proceso de inferencia está limitado a las palabras claves definidas en cada interpretación
- No cuenta con procesamiento ortográfico, los tildes, sinónimos o similares no son asociados a los átomos correspondientes dado la rigidez.
- Ausencia de memoria y falta de entendimiento de contexto, no se pueden realizar preguntas de seguimiento encadenadas.

Uno de los problemas principales del modelo, se presenta al agregar nuevas reglas, varias preguntas específicas eran interceptadas incorrectamente por reglas más genéricas definidas antes en el código. Esto ocurre porque Prolog evalúa las cláusulas de interpretar/2 en el orden en que fueron escritas y, al encontrar la primera que logra unificar, utiliza el operador de corte (!) para descartar las demás alternativas sin seguir buscando una mejor coincidencia. Por ejemplo, la pregunta “¿contra qué tipo de ataque es débil gengar?” era respondida inicialmente con el tipo del pokémon en vez de su debilidad, porque la regla que detecta la palabra “tipo” se evaluaba antes que la regla de debilidad. La solución consistió en reordenar las reglas desde las más específicas (que exigen dos entidades nombradas en la pregunta, como un pokémon y un tipo a la vez) hacia las más genéricas (que solo exigen una palabra clave), de modo que las consultas más precisas se evalúen primero.

Otra limitación observada es la dependencia exacta de las palabras clave: el sistema no reconoce sinónimos/similitudes salvo que se programen explícitamente (como ocurrió con

“contra”, “sobre” y débil en las preguntas de ventaja), ni tolera errores ortográficos en los nombres de los pokémon o entrenadores, dado que la validación se realiza mediante coincidencia exacta de átomos. Asimismo, cada pregunta se procesa de manera independiente, sin memoria de las interacciones previas, por lo que el chatbot no puede responder preguntas de seguimiento como “¿y contra agua?” luego de haber consultado por las debilidades de otro tipo, limitando su conocimiento a la consulta específica.

Pese a estas limitaciones, una vez identificado y corregido los problemas presentados, el chatbot fue capaz de responder correctamente la totalidad de las 20 preguntas planteadas. Esto demuestra que la lógica de primer orden permite modelar relaciones y reglas complejas de forma precisa, aunque el componente de interpretación de lenguaje natural requiere un diseño cuidadoso del orden y la especificidad de las reglas para que dicha precisión se traduzca en respuestas correctas o coherentes según el tipo de consulta.

Conclusiones y mejoras

Gracias a la implementación y aplicación de la LPO, se logró modelar los hechos, reglas y dominio del videojuego *Pokémon* en una base de conocimiento dentro de *SWI-Prolog*, junto a una interfaz conversacional funcional. A pesar de que es limitada a la base de conocimiento actual y las palabras claves de consulta, permite representar de manera ideal un primer acercamiento a un diseño de un chatbot.

Finalmente, para futuras etapas, se buscará la implementación de un LLM via Python para flexibilizar entradas y errores, además de que permita el manejo de errores ortográficos, estandarizando caracteres antes de la tokenización, comprensión adaptada a la consulta, base de conocimiento mucho más amplia para la inclusión de inmunidad o resistencia a los tipos, combinación de condiciones en una pregunta y una memoria de contexto a corto plazo para permitir inferencia en oraciones posteriores. Junto a ello, se busca mitigar las desventajas actuales de este avance, buscando, además, una mejor interacción y aprendizaje del modelo y una interacción con el usuario por medio de un Frontend.

Bibliografía

[1] *Prolog Basics Explained with Pokémon*. (s. f.). Unplanned Obsolescence.

<https://unplannedobsolescence.com/blog/prolog-basics-pokemon/>